

The maze as a tree of branching points

walking it by local moves — a note beside the APP1 report

Mohamed Ali Tewfik Hamilil
M1 Artificial Intelligence, Université Grenoble Alpes

September 12, 2026

Abstract

A proposal: store the maze not as a set of cells but as a binary tree whose nodes hold branching points, give every cell an address in that tree, and solve by walking from the start's node to the exit's, each step moving to whichever of the parent, the children and the sibling lies nearest the exit. This note takes it apart. The *structure* is right and is the one the generator already builds. The *navigation rule* is not: it is greedy best-first search, which is neither optimal nor complete, and on 200 mazes of 32×32 it stopped at a local minimum 199 times. Replacing “nearest the exit” by “which side holds the exit's address” makes the same walk optimal and $\Theta(\log n)$ — measured at $1.06 \log_2 n$ moves. What that buys, and what it does not, is set out at the end.

1 The proposal, stated

Structure.

A binary tree instead of a set of cells. Each internal node stores the **branching point** of the division that created it — the cell its single door sits on. Each leaf is a region. A cell's **address** is the sequence of turns from the root to its leaf.

Solver.

The start and the exit are given as addresses. From the start's node, each step moves to whichever of the **parent**, the **children** and the **sibling** has its stored point nearest the exit, and stops on arrival.

Figure 1 is the structure. It is worth saying at once that this is not a new structure: it is the region tree the generator of the report already returns, with the door recorded at each node.

The sibling move deserves a note of its own. A node's sibling is its parent's other child, so a sibling move is a parent move followed by a child move — a shortcut, not new reach. It matters anyway, because in the maze two sibling regions are joined by exactly one door, their parent's. **A sibling move is the only one of the three that corresponds to walking through a door.**

2 Is it optimal? No, and it is not even complete

The rule is **greedy best-first search**: it ranks moves by an estimate of the remaining distance and ignores the distance already travelled. Greedy best-first is not optimal, and without a closed list it is not complete.

Measured on 200 mazes per size, start at one corner, exit at the other.

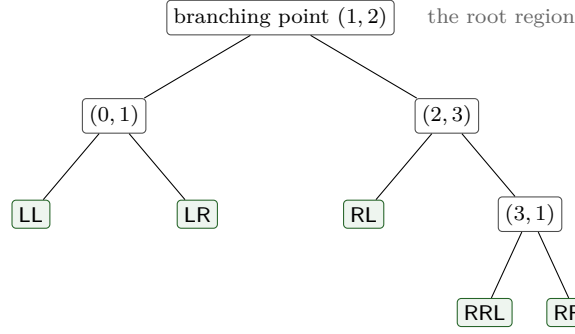


Figure 1: The proposed structure. Each internal node stores the branching point of the division that created it — the cell its single door sits on — and each leaf is a region of the maze. A cell’s **address** is the sequence of turns from the root to its leaf, so **RRL** names one region in three characters. Sibling leaves are the two halves a single division separated, and are joined in the maze by that division’s door alone.

Size	Variant	Reached the exit	Never terminated	Stuck
16×16	as proposed	13	187	—
16×16	with a visited set	24	—	176
32×32	as proposed	6	194	—
32×32	with a visited set	7	—	193
64×64	as proposed	2	198	—
64×64	with a visited set	3	—	197

At 64×64 the walk arrives *once in a hundred*. Adding a visited set converts non-termination into an honest failure, which is an improvement in diagnosis and none at all in outcome.

2.1 The failure mode, diagnosed

Instrumenting the 32×32 runs separates two possible causes and finds only one of them.

Cause	Runs
Stuck at a local minimum: no available move is nearer the exit	199
Cycled: revisited a node already seen	0

So it is not oscillation. It is hill-climbing meeting a hill (Figure 2).

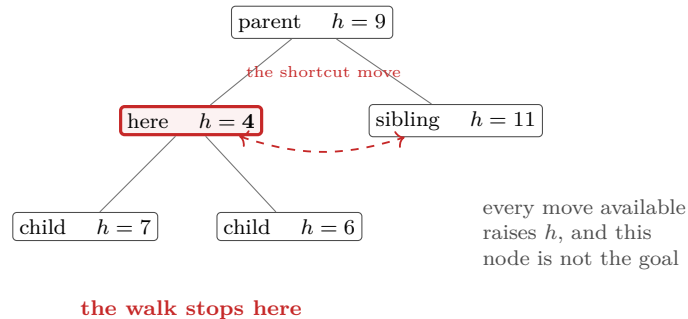


Figure 2: A local minimum, which is where the greedy rule ends 199 runs in 200. The stored branching point of the current node happens to lie nearer the exit than those of its parent, its two children and its sibling; no move improves h , and the exit is elsewhere. Nothing about the maze is wrong — the rule is simply hill-climbing on a landscape with hills in it.

2.2 Why the landscape has hills

Two reasons, and the second is the one specific to this structure.

Greedy search discards g .

Ranking only by the estimate to the goal means a step that walks into a cul-de-sac is taken whenever the cul-de-sac points the right way. This is generic and is why greedy best-first is not used when optimality matters.

A node's stored point is not where the walker is.

An internal node stands for a *region*, not a position; its branching point is one cell inside it, chosen by the generator for reasons unrelated to the exit. Moving to a parent does not move the walker at all — the parent covers the same ground and more. Ranking parents, children and siblings by a single stored cell compares objects that are not comparable.

The estimate answers “how far is this node’s door from the exit”. The walk needs “which way is the exit from here”. Those are different questions, and the tree can answer the second exactly.

3 Time complexity of the proposal

As stated, with no memory: **unbounded**. The walk has no termination argument, and on the evidence above it usually does not terminate. A step cap is the only thing that ends it.

With a visited set: it terminates, because the tree is finite and no node is entered twice, so it performs at most $O(L)$ moves on a tree of L leaves, which is $O(n)$. That is a bound on *giving up*, not on succeeding.

Both figures are worse than Dijkstra’s $\Theta(n)$, which at least always returns the right answer. A cheaper wrong answer is not a better algorithm.

4 The rule that works: navigate by address

The structure was never the problem. Replace the ranking.

Algorithm 1: From the start’s leaf to the exit’s, by address.

Input: the leaf holding the start, and the exit cell.

Output: the leaf holding the exit.

```
1 Function WalkToExitLeaf( $leaf, v_{exit}$ ):  
2    $R \leftarrow leaf$ ;  
3   while  $R$  does not contain  $v_{exit}$  do  
4      $R \leftarrow R.parent$ ;           // climb to the lowest common ancestor  
5   while  $R$  is not a leaf do  
6      $R \leftarrow$  whichever child contains  $v_{exit}$  ;           // descend by address  
7   return  $R$ ;
```

No distance is consulted. Each test is a comparison of the exit’s coordinates against a rectangle, and the walk is forced at every step: up while the region is too small to contain the exit, then down the one branch that does.

Proposition 1. *Algorithm 1 reaches the exit’s leaf, and does so in the fewest parent/child moves possible.*

Proof. Climbing terminates: the root contains every cell. The first node reached that contains the exit is the lowest common ancestor of the two leaves, since every node on the way up was too small and ancestors only grow. Descending is deterministic — exactly one child contains the exit — and ends at its leaf. The moves made are $\text{depth}(\text{start}) - \text{depth}(\text{lca})$ up and $\text{depth}(v_{\text{exit}}) - \text{depth}(\text{lca})$ down, which is the length of the unique path between the two leaves in the tree, and no walk between them can be shorter. $\square$

Measured over 200 mazes per size, against the optimal tree path each time:

n	Moves	$\log_2 n$	Moves / $\log_2 n$	Optimal every run?
256	8.4	8	1.05	yes
1024	10.5	10	1.05	yes
4096	12.8	12	1.07	yes
16384	14.9	14	1.06	yes
65536	17.1	16	1.07	yes

$\Theta(\log n)$, at about $1.06 \log_2 n$ moves, and optimal on every run of every size. The greedy rule reached the exit 1% of the time; the address rule reaches it always, in a number of moves bounded by the depth of the tree.

5 What this actually gives, and what it does not

One qualification matters, and it is easy to miss.

Algorithm 1 finds the **leaf**, not the **route**. It locates the exit's region and, on the way, the lowest common ancestor — whose door is the first door the route crosses. It does not enumerate the rest.

The three costs separate cleanly, and only the first is logarithmic.

Question	Cost	How
Which region holds the exit? Which door first?	$\Theta(\log n)$	Algorithm 1
How many doors between start and exit?	$\Theta(n^{0.68})$	the recursive decomposition
Which cells, in order?	$\Theta(d)$	the size of the answer itself

The middle row is the subject of the companion note; the exponent is measured, and comes from a branching recurrence whose one parameter is the probability that a division separates the two endpoints.

6 Verdict

The structure: keep it.

A binary tree of branching points with addressed leaves is the right representation, and the generator already produces it at no extra cost. It is also what the subject's learning objectives ask to be used.

The navigation rule: replace it.

Ranking parent, children and sibling by distance to the exit is greedy best-first search on an estimate attached to the wrong object. It is not optimal, not complete, and succeeded on 1–12% of runs.

What replaces it is simpler, not harder.

An address comparison, no arithmetic, no estimate, optimal by construction and $\Theta(\log n)$.

The instinct behind the proposal — that the tree already knows where the exit is, and the solver should ask it rather than search the cells — is right, and is what both these notes are about. It is only the question put to the tree that needed changing.